KU LEUVEN

Natural Language Processing: Language modelling

H02B1A



Miryam de Lhoneux

Slides adapted from Jurafsky and Martin (2025)

Department of Computer Science
KU Leuven

Outline

Introduction

N-grams

Generation: sampling

Evaluation

Smoothing

Summary

Language modelling

Predict the next word

Unicorns bring joy and wonder to everyone they ...

- ✅ meet
- ✅ know
- ✅ see
- ❌ the
- ❌ day

Language modelling

Language models

- Can assign a probability to each potential next word
- Can assign a probability to a whole sentence

Language modelling

Why is it useful?

- Grammar or spell checking:

  ~~Their~~ There are two birds

  Everything has ~~improve~~ improved

- Speech recognition:

  I will be bassoon dish $\rightarrow$ I will be back soonish

- It's how large language models (LLMs) work! *(see lecture 4)*

  LLMs are trained to predict words

  LLMs generate text by predicting words

Outline

Introduction

N-grams

Generation: sampling

Evaluation

Smoothing

Summary

Language modelling

Two LM prediction goals

- Probability of a sentence or sequence of words W:

$$P(W) = P(w_1, w_2, w_3, w_4, w_5..., w_n)$$

- Probability of an upcoming word:

$$P(w_5|w_1, w_2, w_3, w_4)$$
$$P(w_n|w_1, w_2, ..., w_{n-1})$$

How can we estimate these probabilities?

Can we just count and divide?

$$P(\text{meet}|\text{Unicorns bring joy and wonder to everyone they}) = \frac{C(\text{Unicorns bring joy and wonder to everyone they meet})}{C(\text{Unicorns bring joy and wonder to everyone they})}$$

No! Estimated on a corpus

Too many possible sentences

$\Rightarrow$ We will not see enough data for reliable estimates

Let's rely on the chain rule of probability

Chain rule

The chain rule of probabilities expresses the joint probability of a sequence of events as the product of each event's conditional probability given the preceding events

Chain rule

$$P(B|A) = \frac{P(A,B)}{P(A)} \qquad \textit{conditional probabilities}$$

$$P(A,B) = P(A)P(B|A) \qquad \textit{rewritten as joint probabilities}$$

Generalized to more variables:

$$P(A,B,C,D) = P(A)P(B|A)P(C|A,B)P(D|A,B,C)$$

General chain rule:

$$P(x_1,x_2,x_3,\cdots,x_n) = P(x_1)P(x_2|x_1)P(x_3|x_1,x_2)\cdots P(x_n|x_1,\cdots,x_{n-1})$$

Chain rule applied to joint probability of words in sentence

$$P(w_{1:n}) = P(w_1)P(w_2|w_1)P(w_3|w_{1:2})...P(w_n|w_{1:n-1})$$
$$= \prod_{k=1}^{n} P(w_k|w_{1:k-1})$$

$$\begin{aligned} P(\text{Unicorns bring joy and wonder}) = & P(\text{Unicorns}) \times P(\text{bring}|\text{Unicorns}) \\ & \times P(\text{joy}|\text{Unicorns bring}) \\ & \times P(\text{and}|\text{Unicorns bring joy}) \\ & \times P(\text{wonder}|\text{Unicorns bring joy and}) \end{aligned}$$

Markov assumption

Before

$P(\text{Unicorns bring joy and wonder}) =$

$P(\textbf{Unicorns})$

$\cdot\ P(\textbf{bring} \mid \text{Unicorns})$

$\cdot\ P(\textbf{joy} \mid \text{Unicorns bring})$

$\cdot\ P(\textbf{and} \mid \text{Unicorns bring joy})$

$\cdot\ P(\textbf{wonder} \mid \text{Unicorns bring joy and})$

After (3-gram)

$P(\text{Unicorns bring joy and wonder}) =$

$P(\textbf{Unicorns}) \longrightarrow P(\textbf{Unicorns})$

$\cdot\ P(\textbf{bring} \mid \text{Unicorns}) \longrightarrow \cdot\ P(\textbf{bring} \mid \text{Unicorns})$

$\cdot\ P(\textbf{joy} \mid \text{Unicorns bring}) \longrightarrow \cdot\ P(\textbf{joy} \mid \text{Unicorns bring})$

$\cdot\ P(\textbf{and} \mid \text{Unicorns bring joy}) \longrightarrow \cdot\ P(\textbf{and} \mid \text{bring joy})$

$\cdot\ P(\textbf{wonder} \mid \underbrace{\text{Unicorns bring}}_{\text{ignore}}\ \underbrace{\text{joy and}}_{\text{use}}) \longrightarrow \cdot\ P(\textbf{wonder} \mid \text{joy and})$

Adapted from 🔗 with the help of Gemini.

Markov assumption

$$P(\text{wonder}|\text{Unicorns bring joy and }) \approx P(\text{wonder}|\text{and}) \quad \text{bigram}$$

$$P(w_n) \approx P(w_n|w_{n-1})$$

$$P(w_{1:n}) \approx \prod_{k=1}^{n} P(w_k|w_{k-1})$$

$$P(w_n|w_{1:n}) \approx P(w_n|w_{n-N+1:n-1}) \quad \text{N-gram}$$

where $N = 2$ for bigrams, $N = 3$ for trigrams, etc.

Unigram model

$$P(w_1, ..., w_n) = \prod_i P(w_i)$$

$$P(w_i|w_{1:n}) = P(w_i)$$

Automatically generated sentences from unigram models

To him swallowed confess hear both . Which . Of save on trail for are ay device and rote life have

Hill he late speaks ; or ! a more to leg less first you enter

Bigram model

$$P(w_1, ..., w_n) = \prod_{k=1}^{n} P(w_k | w_{k-1})$$

$$P(w_i | w_{1:n}) = P(w_i | w_{i-1})$$

Automatically generated sentences from bigram models

Why dost stand forth thy canopy, forsooth; he is this palpable hit the King Henry. Live king. Follow.
What means, sir. I confess she? then all sorts, he is trim, captain.

Maximum Likelihood Estimate (MLE)

- estimates parameters of a model M from a training set T
- maximizes the likelihood of T given M

Intuition

- Suppose the word *bagel* occurs 400 times in a corpus of a million words
- What is the probability that a random word from some other text will be *bagel*?
- MLE estimate: $\frac{400}{1{,}000{,}000} = 0.0004$
- This may be a bad estimate for some other corpus,

  but it is the estimate that makes it most likely that *bagel* occurs 400 times in a million word corpus.

MLE of bigrams

$$P(w_n|w_{1:n}) = P(w_n|w_{n-1})$$

$$P(w_n|w_{n-1}) = \frac{C(w_{n-1}, w_n)}{\sum_w C(w_{n-1}, w)}$$

$$= \frac{C(w_{n-1}, w_n)}{C(w_{n-1})}$$

Dealing with scale

- Multiplying many small numbers may result in underflow

$\Rightarrow$ Use of log probabilities:

$$\log(p_1 \times p_2 \times p_3 \times p_4) = \log p_1 + \log p_2 + \log p_3 + \log p_4$$

- If we need probabilities, use *exp*:

$$p_1 \times p_2 \times p_3 \times p_4 = exp(\log p_1 + \log p_2 + \log p_3 + \log p_4)$$

Limitations of *N*-gram models

- Long-distance dependencies:

  The soups that I made from that new cookbook I bought yesterday were amazingly delicious.

- Meaning: the probabilities of the words *cat* and *dog* are estimated independently.

Solution: (large) language models

- Can handle long context
- Similar words are represented similarly $\Rightarrow$ some meaning is captured

See lecture 4

Outline

Introduction

N-grams

Generation: sampling

Evaluation

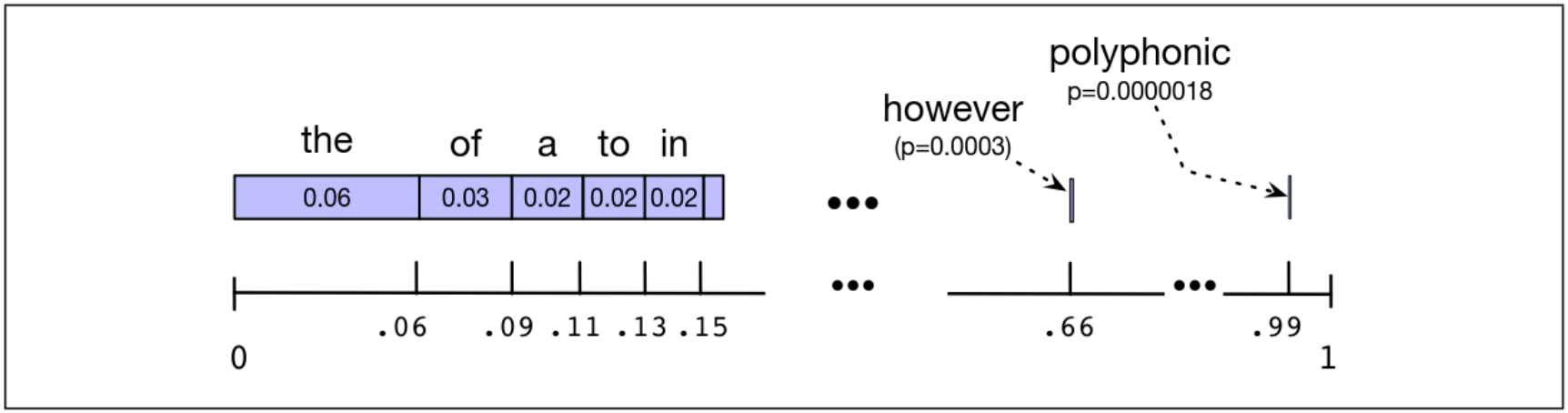
Smoothing

Summary

Sampling words by Shannon (1948)

"Open a book at random and select a letter at random on the page. This letter is recorded. The book is then opened to another page and one reads until this letter is encountered. The succeeding letter is then recorded. Turning to another page this second letter is searched for and the succeeding letter recorded, etc."

Sampling



Visualization of LM sampling according to probabilities (Jurafsky and Martin, 2025).

Generating text using a bigram model

Choose a random bigram $(<\text{s}>, w)$ according to its probability $p(w|<\text{s}>)$

Repeat:

Choose a random bigram (w, x) according to its probability $p(x|w)$

Until a bigram ending with $</\text{s}>$ is selected

Generation

Choose a random bigram (<s>, w) according to its P $p(w|\text{<s>})$

Choose a random bigram (w, x) according to its probability $p(x|w)$

<s> Unicorns
<s> Unicorns bring
<s> Unicorns bring joy
<s> Unicorns bring joy and
<s> Unicorns bring joy and wonder

Until a bigram ending with </s> is selected

<s> Unicorns bring joy and wonder </s>

Other sampling methods

Many avoid generating words from the unlikely tail of the distribution.

- Temperature sampling (Ficler and Goldberg, 2017)
- Top-k sampling (Fan et al., 2018)
- Top-p sampling (Holtzman et al., 2020)

See lecture 4.

1 gram	–To him swallowed confess hear both. Which. Of save on trail for are ay device and rote life have –Hill he late speaks; or! a more to leg less first you enter
2 gram	–Why dost stand forth thy canopy, forsooth; he is this palpable hit the King Henry. Live king. Follow. –What means, sir. I confess she? then all sorts, he is trim, captain.
3 gram	–Fly, and will rid me these news of price. Therefore the sadness of parting, as they say, 'tis done. –This shall forbid it should be branded, if renown made it empty.
4 gram	–King Henry. What! I will go seek the traitor Gloucester. Exeunt some of the watch. A great banquet serv'd in; –It cannot be but so.

Sentences generated by N-gram models trained on Shakespeare text (Jurafsky and Martin, 2025).

1 gram	Months the my and issue of year foreign new exchange's september were recession exchange new endorsed a acquire to six executives
2 gram	Last December through the way to preserve the Hudson corporation N. B. E. C. Taylor would seem to complete the major central planners one point five percent of U. S. E. has already old M. X. corporation of living on information such as more frequently fishing to keep her
3 gram	They also point to ninety nine point six billion dollars from two hundred four oh six three percent of the rates of interest stores as Mexico and Brazil on market conditions

Sentences generated by *N*-gram models trained on the WSJ (Jurafsky and Martin, 2025).

Outline

Introduction

N-grams

Generation: sampling

Evaluation

Smoothing

Summary

Evaluation

Extrinsic/dowstream evaluation

Comparing LM A to LM B:

1. Use them in a task, e.g. machine translation or speech recognition
2. Evaluate performance on the task
3. Compare task accuries for LM A vs LM B

Limitations

- Expensive and time-consuming
- May not generalize to other applications

Evaluation

Intrinsic evaluation: perplexity

- Directly measures language model performance at predicting words.
- Doesn't necessarily correspond with real application performance
- Gives us a single general metric for language models (including LLMs)

Train vs test set

- Model parameters are trained/estimated on a training set.
- Model performance is measured on an unseen test set using an evaluation metric (here perplexity).

We want to measure generalization to unseen data

Choosing the test set

Use cases:

- Specific task: the test set should reflect the task language (e.g. legal or medical domain)
- General purpose: need for diversity in domains, authors etc.

⚠ Make sure there are no test sentences in the training set.

Evaluation

Dev set

- If we repeatedly test on the test set, we might tune the model to its characteristics

$\Rightarrow$ Keep it for final evaluation

- Use a development set for intermediate evaluations

Evaluation

Perplexity

Intuition: A good LM prefers "real" sentences.

- Assigns higher probability to "real" or "frequently observed" sentences
- Assigns lower probability to "word salad" or "rarely observed" sentences

Evaluation

Perplexity

Intuition: A good LM can predict upcoming words.

- Once upon a ____
- I ordered a pizza with cheese and ____
- I saw a ____

time	0.9
dream	0.03
midnight	0.02
...	
giraffe	0.0001
...	
and	1e-100

Evaluation

The best LM assigns high probability to the entire test set.
Comparing LM A to LM B:

- Compute and compare $P_A(\text{test set})$ and $P_B(\text{test set})$
- The better LM has the higher probability

$\rightarrow$ It is less surprised by the test set

Evaluation

Why not use probability?

The probability of a test set depends on its size (longer = less probable)

$\Rightarrow$ Need for a per-word metric, normalized by length

Evaluation

Perplexity

Perplexity is the inverse probability of the test set, normalized by the number of words:

$$PP(W) = P(w_1 w_2 ... w_N)^{-\frac{1}{N}}$$

$$= \sqrt[N]{\frac{1}{P(w_1 w_2 ... w_N)}}$$

- Probability range is $[0, 1]$, perplexity range is $[1, \infty]$
- Minimizing perplexity is the same as maximizing probability

Evaluation

Perplexity of bi-gram LMs

$$PP(W) = \sqrt[N]{\frac{1}{P(w_1 w_2 \ldots w_N)}}$$

$$= \sqrt[N]{\prod_{i=1}^{N} \frac{1}{P(w_i | w_1 \ldots w_{i-1})}} \qquad \textit{Chain rule}$$

$$= \sqrt[N]{\prod_{i=1}^{N} \frac{1}{P(w_i | w_{i-1})}} \qquad \textit{Bigrams}$$

Evaluation

Example perplexity scores for the WSJ corpus with 38M words in training and 1.5M words in test.

N-gram order	unigram	bigram	trigram
Perplexity	962	170	109

Outline

Introduction

N-grams

Generation: sampling

Evaluation

Smoothing

Summary

Sparsity

Shakespeare as corpus

- $N = 884K$ tokens, $V = 29K$ types
- $V^2 = 844M$ possible bigrams
- Bigrams actually occurring: $300K$

$\Rightarrow$ 99.96% of the possible bigrams were never seen (have zero entries in the table)

- That sparsity is even worse for 4-grams

Generalization

Training sentences

- ··· ate lunch
- ··· ate dinner
- ··· ate a
- ··· ate the

Test sentences

- ··· ate lunch
- ··· ate breakfast

$$p(\text{breakfast}|\text{ate}) = 0$$

Zero probabilities

- 0 probability bigrams lead to assigning 0 probability to the test set
- Which means that we cannot compute perplexity (division by zero)!

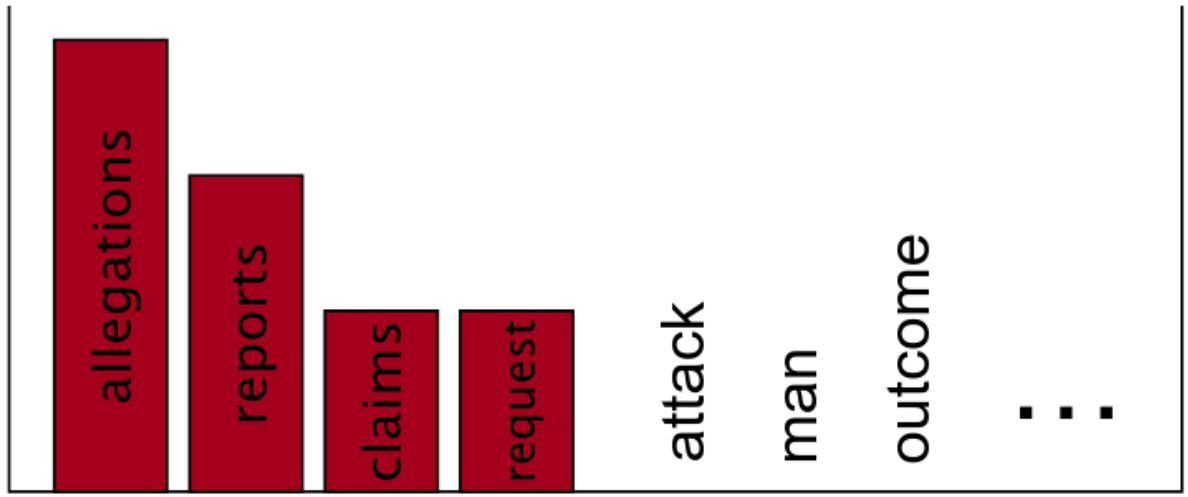
$$PP(W) = \sqrt[N]{\prod_{i=1}^{N} \frac{1}{P(w_i|w_{i-1})}}$$

Smoothing

Sparse statistics

$C(w|\text{denied the})$:

3	allegations
2	reports
1	claims
1	request
7	total



Smoothing illustration adapted from Dan Klein

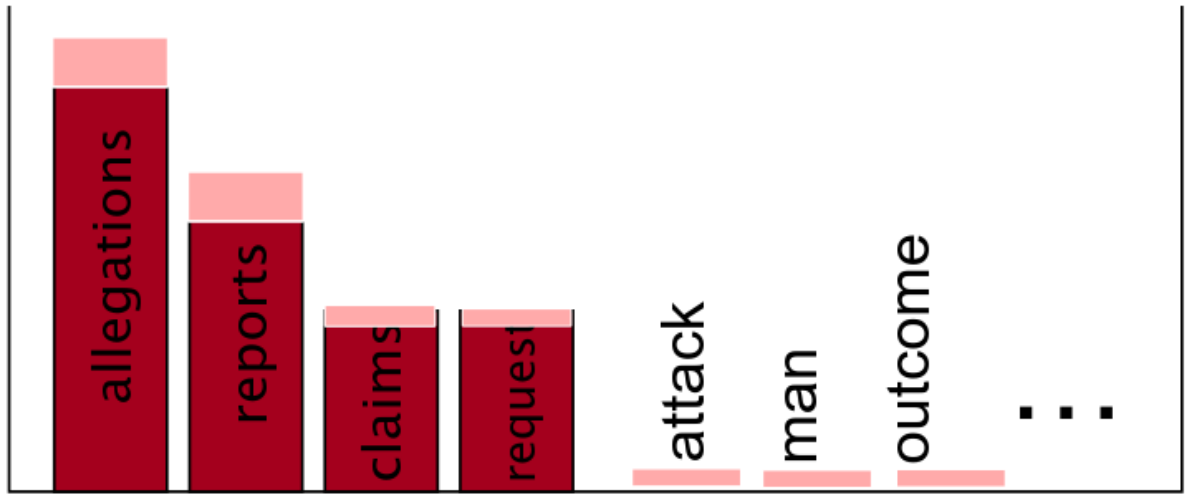
Steal from the rich to give to the poor

Smoothing

Adjusted

$C(w|\text{denied the})$

2.5	allegations
1.5	reports
0.5	claims
0.5	request
2	other
7	total



Smoothing illustration adapted from Dan Klein

Steal from the rich to give to the poor

Smoothing

Add-one/Laplace smoothing

$$\textit{MLE}: \quad P_{MLE}(w_n|w_{n-1}) = \frac{C(w_{n-1}, w_n)}{\sum_w C(w_{n-1}, w)}$$

$$\textit{Laplace}: \quad P_{Laplace}(w_n|w_{n-1}) = \frac{C(w_{n-1}, w_n)+1}{\sum_w (C(w_{n-1}, w)+1)} = \frac{C(w_{n-1}, w_n)+1}{C(w_{n-1}) + V}$$

Smoothing

Add-one smoothing

- Is a blunt instrument: it makes very big changes to the counts

$\Rightarrow$ Is typically not used for N-gram LMs

- Used in NLP applications where the number of zeros is not huge
- Alternative: add-k where $0 < k < 1$, still does not work well

Backoff and Interpolation

Backoff

If you have good evidence: use trigrams

Otherwise bigram

Otherwise unigram

Interpolation

Mix unigram, bigram, trigram

Empirically, interpolation works better.

Backoff

(Stupid) backoff

- Suppose you want:

  P(pancakes|delicous soufflé)

- If the trigram probability is 0, use the bigram

  P(pancakes|soufflé)

- If the bigram probability is 0, use the unigram

  P(pancakes)

Sum of all probabilities of the trigram $> 1 \Rightarrow$ need to discount the higher-order ngram probabilities (e.g., Katz backoff)

Interpolation

Simple interpolation

$$\begin{aligned}\hat{P}(w_n|w_{n-2}w_{n-1}) &= \lambda_1 P(w_n|w_{n-2}w_{n-1}) && \textit{trigram}\\ &+ \lambda_2 P(w_n|w_{n-1}) && \textit{bigram}\\ &+ \lambda_3 P(w_n) && \textit{unigram}\end{aligned}$$

$$\text{where} \sum_i \lambda_i = 1$$

Interpolation

Lambdas conditioned on context

$$\begin{aligned}\hat{P}(w_n|w_{n-2}w_{n-1}) = &\lambda_1(w_{n-2}^{n-1})P(w_n|w_{n-2}w_{n-1}) \\ &+ \lambda_2(w_{n-2}^{n-1})P(w_n|w_{n-1}) \\ &+ \lambda_3(w_{n-2}^{n-1})P(w_n)\end{aligned}$$

Interpolation

Interpolation: choosing λs

- Fix the N-gram probabilities (on the training data)
- Then search for λ s that give largest probability to held-out set

Out of vocabulary words

- Smoothing does not solve the problem of words unseen in the training data
- Out-of-vocabulary (OOV) words (e.g. *whale* in the exercise)
- In practice, low frequency words are replaced by an <UNK> token
- Low-frequency words and unseen words are estimated based on <UNK>

Outline

Introduction

N-grams

Generation: sampling

Evaluation

Smoothing

Summary

Summary and outlook

N-gram language models

- Estimation with MLE
- Generation (sampling)
- Evaluation with perplexity
- Choice of training/test data for generalization
- Limitations of MLE and workarounds

Summary and outlook

Outlook: LLMs

- Estimation and generation with neural networks
- LLMs capture meaning and can handle long-distance dependencies
- Evaluated with (modified) perplexity
- Generalization concerns remain

See lecture 4!

References

Angela Fan, Mike Lewis, and Yann Dauphin. 2018. Hierarchical neural story generation. In *Proceedings of the 56th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*.

Jessica Ficler and Yoav Goldberg. 2017. Controlling linguistic style aspects in neural language generation. In *Proceedings of the Workshop on Stylistic Variation*.

Ari Holtzman, Jan Buys, Li Du, Maxwell Forbes, and Yejin Choi. 2020. The curious case of neural text degeneration. In *International Conference on Learning Representations*.

Daniel Jurafsky and James H. Martin. 2025. *Speech and Language Processing: An Introduction to Natural Language Processing, Computational Linguistics, and Speech Recognition, with Language Models*, 3rd edition. Online manuscript released August 24, 2025.

Claude Elwood Shannon. 1948. A mathematical theory of communication. *The Bell system technical journal*, 27(3):379–423.